

```
// File: flutter_app/lib/main.dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'app_theme.dart';
import 'main_shell.dart';
import 'startup_flow.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
  SystemChrome.setSystemUIOverlayStyle(
    const SystemUiOverlayStyle(
      statusBarColor: Colors.transparent,
      statusBarIconBrightness: Brightness.dark,
      statusBarBrightness: Brightness.light,
      systemNavigationBarColor: Colors.transparent,
      systemNavigationBarIconBrightness: Brightness.dark,
      systemNavigationBarDividerColor: Colors.transparent,
    ),
  );
  runApp(const UzBoxApp());
}

class UzBoxApp extends StatelessWidget {
  const UzBoxApp({super.key, this.skipStartup = false});
  final bool skipStartup;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'UzBox',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.light,
        fontFamily: '.SF Pro Text',
        scaffoldBackgroundColor: const Color(0xFF8FAFA),
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.black,
          brightness: Brightness.light,
          surface: Colors.white,
        ),
        splashFactory: NoSplash.splashFactory,
        highlightColor: Colors.transparent,
        textTheme: ThemeData.light().textTheme.apply(
          bodyColor: AppTheme.primary,
          displayColor: AppTheme.primary,
        ),
      ),
      home: skipStartup ? const MainShell() : const StartupFlow(),
    );
  }
}
```

```
// File: flutter_app/lib/startup_flow.dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'app_theme.dart';
import 'main_shell.dart';
class StartupFlow extends StatefulWidget {
  const StartupFlow({super.key});

  @override
  State<StartupFlow> createState() => _StartupFlowState();
}
class _StartupFlowState extends State<StartupFlow>
  with SingleTickerProviderStateMixin {
  late final AnimationController transitionController;
  bool showsSplash = true;
  bool showsMain = false;
  @override
  void initState() {
    super.initState();
    transitionController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1150),
    );
  }
  @override
  void dispose() {
    transitionController.dispose();
    super.dispose();
  }
  Future<void> _enterMain() async {
    setState(() => showsMain = true);
    await transitionController.forward(from: 0);
  }
  @override
  Widget build(BuildContext context) {
    return Stack(
      fit: StackFit.expand,
      children: [
        if (showsMain)
          const MainShell()
        else if (!showsSplash)
          LoginScreen(onLogin: _enterMain),
        if (showsSplash)
          AppSplashScreen(onFinish: () => setState(() => showsSplash = false)),
        if (showsMain && transitionController.value < 1)
          _HomeWordmarkTransition(animation: transitionController),
      ],
    );
  }
}
class AppSplashScreen extends StatefulWidget {
```

```

const AppSplashScreen({super.key, required this.onFinish});
final VoidCallback onFinish;

@override
State<AppSplashScreen> createState() => _AppSplashScreenState();
}

class _AppSplashScreenState extends State<AppSplashScreen> {
  bool visible = false;
  bool exiting = false;

  @override
  void initState() {
    super.initState();
    scheduleMicrotask(() => setState(() => visible = true));
    unawaited(_run());
  }

  Future<void> _run() async {
    await Future<void>.delayed(const Duration(milliseconds: 1300));
    if (!mounted) return;
    setState(() => exiting = true);
    await Future<void>.delayed(const Duration(milliseconds: 320));
    if (mounted) widget.onFinish();
  }

  @override
  Widget build(BuildContext context) {
    final compact = MediaQuery.sizeOf(context).height < 730;
    return Material(
      color: Colors.white,
      child: SafeArea(
        child: AnimatedOpacity(
          opacity: exiting ? 0 : 1,
          duration: const Duration(milliseconds: 320),
          curve: Curves.easeInOut,
          child: AnimatedScale(
            scale: visible ? (exiting ? 1.035 : 1) : .88,
            duration: const Duration(milliseconds: 550),
            curve: Curves.easeOutCubic,
            child: Column(
              children: [
                Spacer(flex: compact ? 3 : 4),
                Text(
                  'Uz',
                  style: TextStyle(
                    color: AppTheme.primary,
                    fontSize: compact ? 116 : 154,
                    height: .88,
                    fontWeight: FontWeight.w900,
                    letterSpacing: compact ? -12 : -16,
                  ),
                ),
                SizedBox(height: compact ? 18 : 28),
                Text(

```

```

        'UzBox',
        style: TextStyle(
          color: AppTheme.primary,
          fontSize: compact ? 32 : 38,
          fontWeight: FontWeight.w900,
        ),
      ),
      const SizedBox(height: 8),
      Text(
        'AI 装机助手',
        style: TextStyle(
          color: const Color(0xFF474747),
          fontSize: compact ? 17 : 19,
          fontWeight: FontWeight.w500,
        ),
      ),
      const SizedBox(height: 10),
      Text(
        '智能推荐最佳配置方案，让装机更简单',
        style: TextStyle(
          color: const Color(0xFF9E9E9E),
          fontSize: compact ? 12 : 14,
        ),
      ),
      const Spacer(flex: 4),
    ],
  ),
),
),
),
);
}

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key, required this.onLogin});
  final VoidCallback onLogin;

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  bool accepted = false;
  bool isAuthenticating = false;
  Future<void> _login() async {
    if (!accepted || isAuthenticating) return;
    setState(() => isAuthenticating = true);
    await Future<void>.delayed(const Duration(milliseconds: 520));
    if (mounted) widget.onLogin();
  }

  @override
  Widget build(BuildContext context) {

```

```
final isIos = Theme.of(context).platform == TargetPlatform.iOS;
return Material(
  color: Colors.white,
  child: SafeArea(
    child: LayoutBuilder(
      builder: (context, constraints) {
        final compact = constraints.maxHeight < 730;
        final horizontal = (constraints.maxWidth * .075).clamp(24.0, 34.0);
        return SingleChildScrollView(
          physics: const BouncingScrollPhysics(),
          padding: EdgeInsets.fromLTRB(
            horizontal,
            compact ? 10 : 18,
            horizontal,
            18,
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'UzBox',
                style: TextStyle(
                  color: AppTheme.primary,
                  fontSize: compact ? 23 : 26,
                  fontWeight: FontWeight.w900,
                  letterSpacing: -1,
                ),
              ),
              SizedBox(height: compact ? 4 : 10),
              Center(
                child: ConstrainedBox(
                  constraints: const BoxConstraints(maxWidth: 420),
                  child: Image.asset(
                    'assets/images/login_hardware_hero.png',
                    height: compact ? 245 : 310,
                    fit: BoxFit.contain,
                  ),
                ),
              ),
              SizedBox(height: compact ? 4 : 8),
              Text(
                '更聪明地\n装好一台电脑',
                style: TextStyle(
                  color: AppTheme.primary,
                  fontSize: compact ? 33 : 36,
                  height: compact ? 1.06 : 1.13,
                  fontWeight: FontWeight.w900,
                  letterSpacing: -1.2,
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  ),
)
```

```
SizedBox(height: compact ? 10 : 14),
const Text(
  '从需求到配置，UzBox 帮你做出更稳妥的选择',
  maxLines: 1,
  overflow: TextOverflow.fade,
  style: TextStyle(color: Color(0xFF858585), fontSize: 14),
),
SizedBox(height: compact ? 42 : 58),
Center(
  child: ConstrainedBox(
    constraints: const BoxConstraints(maxWidth: 320),
    child: SizedBox(
      width: double.infinity,
      height: compact ? 54 : 58,
      child: FilledButton.icon(
        onPressed: accepted ? _login : _showConsentReminder,
        icon: isAuthenticating
          ? const SizedBox(
              width: 19,
              height: 19,
              child: CircularProgressIndicator(
                color: Colors.white,
                strokeWidth: 2,
              ),
            )
          : Icon(
              islos
                ? Icons.apple_rounded
                : Icons.phone_iphone_rounded,
              size: islos ? 24 : 22,
            ),
        label: Text(
          isAuthenticating
            ? '正在登录'
            : islos
              ? '通过 Apple 登录'
              : '本机号码一键登录',
          style: const TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.w700,
          ),
        ),
      ),
      style: FilledButton.styleFrom(
        backgroundColor: Colors.black,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(17),
        ),
      ),
    ),
  ),
)
```

```

    ),
  ),
  SizedBox(height: compact ? 14 : 18),
  Center(
    child: Text(
      islos ? '使用 Apple 账号安全登录' : '由运营商提供本机号码认证服务',
      style: const TextStyle(
        color: Color(0xFF9A9A9A),
        fontSize: 11,
      ),
    ),
  ),
  ),
  const SizedBox(height: 8),
  Center(
    child: InkWell(
      onTap: () => setState(() => accepted = !accepted),
      borderRadius: BorderRadius.circular(10),
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: 5),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Container(
              width: 20,
              height: 20,
              decoration: BoxDecoration(
                color: accepted ? Colors.black : Colors.white,
                shape: BoxShape.circle,
                border: Border.all(
                  color: accepted
                    ? Colors.black
                    : const Color(0xFFB9B9B9),
                ),
            ),
            child: accepted
              ? const Icon(
                  Icons.check_rounded,
                  color: Colors.white,
                  size: 14,
                )
              : null,
          ),
        ),
        const SizedBox(width: 9),
        const Text(
          '我已阅读并同意《用户协议》和《隐私政策》',
          style: TextStyle(
            color: Color(0xFF737373),
            fontSize: 12,
          ),
        ),
      ),
    ),
  ),

```

```

        ],
      ),
    ),
  ),
],
),
);
},
),
),
);
}

void _showConsentReminder() {
  showDialog<void>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('请先阅读并同意'),
      content: const Text('请先阅读并同意用户协议和隐私政策。'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('知道了'),
        ),
      ],
    ),
  );
}

class _HomeWordmarkTransition extends StatelessWidget {
  const _HomeWordmarkTransition({required this.animation});
  final Animation<double> animation;
  @override
  Widget build(BuildContext context) {
    return IgnorePointer(
      child: AnimatedBuilder(
        animation: animation,
        builder: (context, _) {
          final size = MediaQuery.sizeOf(context);
          final safeTop = MediaQuery.paddingOf(context).top;
          final curved = Curves.easeInOutCubicEmphasized.transform(
            animation.value,
          );
          final left = Tween<double>(
            begin: size.width / 2 - 54,
            end: 22,
          ).transform(curved);
          final top = Tween<double>(
            begin: size.height / 2 - 24,
            end: safeTop + 12,

```



```

    ).transform(curved);
    final fontSize = Tween<double>{(begin: 38, end: 28).transform(curved);
    final opacity = animation.value < .78
      ? 1.0
      : (1 - (animation.value - .78) / .22).clamp(0.0, 1.0);
    return Stack(
      children: [
        Positioned.fill(
          child: ColoredBox(
            color: Colors.white.withValues(alpha: (1 - curved) * .88),
          ),
        ),
        Positioned(
          left: left,
          top: top,
          child: Opacity(
            opacity: opacity,
            child: Text(
              'UzBox',
              style: TextStyle(
                color: AppTheme.primary,
                fontSize: fontSize,
                fontWeight: FontWeight.w900,
              ),
            ),
          ),
        ),
      ],
    );
  },
),
);
}
}

// File: flutter_app/lib/screens/home_screen.dart
import 'dart:math' as math;
import 'package:flutter/material.dart';
import '../app_theme.dart';
class _HomeFeature {
  const _HomeFeature({
    required this.title,
    required this.subtitle,
    required this.action,
    required this.bullets,
    required this.image,
    required this.icon,
    required this.imageWidth,
    required this.imageHeight,
    this.titleSize = 34,
    this.textWidth = 224,

```

```
});  
final String title;  
final String subtitle;  
final String action;  
final List<String> bullets;  
final String image;  
final IconData icon;  
final double imageWidth;  
final double imageHeight;  
final double titleSize;  
final double textWidth;  
}  
class _BuildStyle {  
  const _BuildStyle(this.title, this.cost, this.image);  
  final String title;  
  final String cost;  
  final String image;  
}  
class HomeScreen extends StatefulWidget {  
  const HomeScreen({super.key, this.onOpenFeature, this.onOpenStyle});  
  final ValueChanged<int>? onOpenFeature;  
  final ValueChanged<int>? onOpenStyle;  
  @override  
  State<HomeScreen> createState() => _HomeScreenState();  
}  
class _HomeScreenState extends State<HomeScreen> {  
  int _selectedFeature = 0;  
  static const _features = <_HomeFeature>[  
    _HomeFeature(  
      title: 'AI 一键装机',  
      subtitle: '智能推荐装机方案',  
      action: '开始装机',  
      bullets: ['智能推荐配置', '自动检测兼容性', '优化预算方案'],  
      image: 'assets/images/home_style_black_knight.png',  
      icon: Icons.handyman_outlined,  
      imageWidth: 167,  
      imageHeight: 266,  
      textWidth: 213,  
    ),  
    _HomeFeature(  
      title: '游戏性能测试',  
      subtitle: '检测游戏帧率表现',  
      action: '开始测试',  
      bullets: ['帧率表现评估', '硬件瓶颈分析', '游戏场景建议'],  
      image: 'assets/images/home_hero_performance_gpu.png',  
      icon: Icons.sports_esports_outlined,  
      imageWidth: 220,  
      imageHeight: 300,  
      titleSize: 27,  
      textWidth: 160,
```

```
),
  _HomeFeature(
    title: '配置排雷',
    subtitle: '判断配置能不能买',
    action: '开始排雷',
    bullets: ['识别搭配风险', '检查兼容问题', '提示预算浪费'],
    image: 'assets/images/home_hero_config_review_board.png',
    icon: Icons.verified_user_outlined,
    imageWidth: 146,
    imageHeight: 236,
    textWidth: 213,
  ),
  _HomeFeature(
    title: '升级建议',
    subtitle: '按预算给出升级顺序',
    action: '查看建议',
    bullets: ['定位升级短板', '排序更换优先级', '匹配预算方案'],
    image: 'assets/images/home_hero_upgrade_parts.png',
    icon: Icons.north_east_rounded,
    imageWidth: 162,
    imageHeight: 232,
    textWidth: 213,
  ),
];

static const _styles = <_BuildStyle>[
  _BuildStyle(
    '联立 VISION COMPACT',
    '外观方案约 ¥2,735 起',
    'assets/images/style_vision_compact_black.png',
  ),
  _BuildStyle(
    'ROG 创世神 701',
    '外观方案约 ¥3,476 起',
    'assets/images/style_rog_gr701_black.png',
  ),
  _BuildStyle(
    '未知玩家 幻翼',
    '外观方案约 ¥1,257 起',
    'assets/images/style_phantom_wing_black.png',
  ),
];

@override
Widget build(BuildContext context) {
  return ColoredBox(
    color: AppTheme.homeBackground,
    child: SafeArea(
      bottom: false,
      child: LayoutBuilder(
        builder: (context, constraints) {
          final width = math.min(constraints.maxWidth - 34, 406.0);
```

```

return SingleChildScrollView(
  physics: const BouncingScrollPhysics(),
  padding: const EdgeInsets.only(bottom: 112),
  child: Transform.translate(
    offset: const Offset(0, -3),
    child: Center(
      child: SizedBox(
        width: width,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Padding(
              padding: EdgeInsets.symmetric(horizontal: 4),
              child: Text(
                'UzBox',
                style: TextStyle(
                  fontFamily: '.SF Pro Display',
                  fontSize: 28,
                  fontWeight: FontWeight.w900,
                  letterSpacing: -0.6,
                ),
              ),
            ),
            const SizedBox(height: 18),
            AnimatedSwitcher(
              duration: const Duration(milliseconds: 260),
              switchInCurve: Curves.easeOut,
              switchOutCurve: Curves.easeIn,
              child: _Hero(
                key: ValueKey(_selectedFeature),
                feature: _features[_selectedFeature],
                onOpen: () =>
                  widget.onOpenFeature?.call(_selectedFeature),
                onSwipe: (delta) {
                  setState(() {
                    _selectedFeature =
                      (_selectedFeature + delta) %
                      _features.length;
                    if (_selectedFeature < 0) {
                      _selectedFeature += _features.length;
                    }
                  });
                },
              ),
            ),
            const SizedBox(height: 24),
            _FeatureSelector(
              selected: _selectedFeature,
              onSelected: (index) =>
                setState(() => _selectedFeature = index),
            ),
          ],
        ),
      ),
    ),
  ),
);

```

```

    ),
    const SizedBox(height: 34),
    const Text(
      '精选装机风格',
      style: TextStyle(
        fontSize: 18,
        height: 1.22,
        fontWeight: FontWeight.w900,
      ),
    ),
    const SizedBox(height: 5),
    Text(
      '找到你喜欢的主机外观与氛围',
      style: TextStyle(
        color: Colors.black.withValues(alpha: 0.52),
        fontSize: 12,
        height: 1.25,
      ),
    ),
    const SizedBox(height: 16),
    ...List.generate(_styles.length, (index) {
      return Column(
        children: [
          _StyleRow(
            style: _styles[index],
            onTap: () => widget.onOpenStyle?.call(index),
          ),
          if (index != _styles.length - 1)
            Divider(
              height: 1,
              thickness: 1,
              color: Colors.black.withValues(alpha: 0.07),
            ),
        ],
      );
    }),
  ],
),
),
),
),
);
},
),
),
);
},
),
);
}
}

class _Hero extends StatelessWidget {
  const _Hero({

```

```
super.key,
required this.feature,
required this.onSwipe,
required this.onOpen,
});
final _HomeFeature feature;
final ValueChanged<int> onSwipe;
final VoidCallback onOpen;
@override
Widget build(BuildContext context) {
  return GestureDetector(
    onHorizontalDragEnd: (details) {
      final velocity = details.primaryVelocity ?? 0;
      if (velocity.abs() > 120) onSwipe(velocity < 0 ? 1 : -1);
    },
    child: SizedBox(
      height: 316,
      child: Stack(
        children: [
          Positioned(
            left: 20,
            top: 31,
            bottom: 20,
            width: feature.textWidth,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  '当前功能',
                  style: TextStyle(
                    color: Colors.black.withValues(alpha: 0.50),
                    fontSize: 13,
                    height: 1.23,
                    fontWeight: FontWeight.w500,
                  ),
                ),
                const SizedBox(height: 16),
                FittedBox(
                  fit: BoxFit.scaleDown,
                  alignment: Alignment.centerLeft,
                  child: Text(
                    feature.title,
                    maxLines: 1,
                    style: TextStyle(
                      fontSize: feature.titleSize,
                      height: 1.12,
                      fontWeight: FontWeight.w900,
                      letterSpacing: -0.8,
                    ),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  ),
);
```

```
    ),  
    const SizedBox(height: 9),  
    FittedBox(  
      fit: BoxFit.scaleDown,  
      alignment: Alignment.centerLeft,  
      child: Text(  
        feature.subtitle,  
        maxLines: 1,  
        style: TextStyle(  
          color: Colors.black.withValues(alpha: 0.70),  
          fontSize: 15,  
          height: 1.20,  
          fontWeight: FontWeight.w500,  
          letterSpacing: 1.8,  
        ),  
      ),  
    ),  
    const SizedBox(height: 30),  
    ...feature.bullets.map(  
      (bullet) => Padding(  
        padding: const EdgeInsets.only(bottom: 9),  
        child: Row(  
          children: [  
            Icon(  
              Icons.check_circle_outline,  
              size: 13,  
              color: Colors.black.withValues(alpha: 0.48),  
            ),  
            const SizedBox(width: 8),  
            Text(  
              bullet,  
              style: TextStyle(  
                color: Colors.black.withValues(alpha: 0.48),  
                fontSize: 13,  
                height: 1.23,  
              ),  
            ),  
          ],  
        ),  
      ),  
    ),  
    const Spacer(),  
    _HeroButton(title: feature.action, onTap: onOpen),  
  ],  
),  
),  
Positioned(  
  right: 6,  
  top: 50,  
  width: feature.imageWidth,
```

```

        height: feature.imageHeight,
        child: Image.asset(feature.image, fit: BoxFit.contain),
      ),
    ],
  ),
),
);
}
}

class _HeroButton extends StatelessWidget {
  const _HeroButton({required this.title, required this.onTap});
  final String title;
  final VoidCallback onTap;
  @override
  Widget build(BuildContext context) {
    return Semantics(
      label: title,
      button: true,
      child: Container(
        width: 136,
        height: 44,
        decoration: BoxDecoration(
          color: Colors.black,
          borderRadius: BorderRadius.circular(22),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withValues(alpha: 0.20),
              blurRadius: 14,
              offset: const Offset(0, 8),
            ),
          ],
        ),
        child: Material(
          color: Colors.transparent,
          child: InkWell(
            borderRadius: BorderRadius.circular(22),
            onTap: onTap,
            child: Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Text(
                  title,
                  style: const TextStyle(
                    color: Colors.white,
                    fontSize: 15,
                    fontWeight: FontWeight.w700,
                  ),
                ),
                const SizedBox(width: 18),
                const Icon(

```



```

        Icons.arrow_forward_rounded,
        color: Colors.white,
        size: 21,
      ),
    ],
  ),
),
),
),
);
}
}

class _FeatureSelector extends StatelessWidget {
  const _FeatureSelector({required this.selected, required this.onSelected});
  final int selected;
  final ValueChanged<int> onSelected;
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 50,
      child: Row(
        children: List.generate(_HomeScreenState._features.length, (index) {
          final isSelected = index == selected;
          return Expanded(
            child: Center(
              child: AnimatedContainer(
                duration: const Duration(milliseconds: 220),
                width: 50,
                height: 50,
                decoration: BoxDecoration(
                  color: isSelected
                    ? Colors.black
                    : Colors.white.withValues(alpha: 0.72),
                  borderRadius: BorderRadius.circular(16),
                  boxShadow: [
                    BoxShadow(
                      color: Colors.black.withValues(
                        alpha: isSelected ? 0.13 : 0.035,
                      ),
                      blurRadius: isSelected ? 16 : 8,
                      offset: const Offset(0, 7),
                    ),
                  ],
                ),
              child: Material(
                color: Colors.transparent,
                child: InkWell(
                  borderRadius: BorderRadius.circular(16),
                  onTap: () => onSelected(index),
                  child: Icon(

```

```

        _HomeScreenState._features[index].icon,
        size: 24,
        color: isSelected ? Colors.white : Colors.black,
      ),
    ),
  ),
),
);
}),
),
);
}
}

class _StyleRow extends StatelessWidget {
  const _StyleRow({required this.style, required this.onTap});
  final _BuildStyle style;
  final VoidCallback onTap;
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 126,
      child: InkWell(
        onTap: onTap,
        child: Row(
          children: [
            Expanded(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    style.title,
                    maxLines: 1,
                    style: const TextStyle(
                      fontSize: 18,
                      height: 1.22,
                      fontWeight: FontWeight.w700,
                    ),
                  ),
                  const SizedBox(height: 7),
                  Text(
                    style.cost,
                    style: TextStyle(
                      color: Colors.black.withValues(alpha: 0.72),
                      fontSize: 13,
                      height: 1.23,
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        const SizedBox(height: 7),
        const Text(
          '按这个风格装机 →',
          style: TextStyle(
            color: AppTheme.secondary,
            fontSize: 12,
            height: 1.25,
            fontWeight: FontWeight.w500,
          ),
        ),
      ],
    ),
    Transform.translate(
      offset: const Offset(12, 0),
      child: SizedBox(
        width: 148,
        height: 106,
        child: Image.asset(style.image, fit: BoxFit.contain),
      ),
    ),
  ],
),
);
}
}

// File: flutter_app/lib/api/uzbox_api.dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'dart:typed_data';
class UzBoxApiException implements Exception {
  const UzBoxApiException(this.message, {this.statusCode});
  final String message;
  final int? statusCode;
  @override
  String toString() => message;
}

class UzBoxApi {
  UzBoxApi({Uri? baseUrl, HttpClient? client})
    : baseUrl =
      baseUrl ??
      Uri.parse(
        const String.fromEnvironment(
          'UZBOX_API_BASE_URL',
          defaultValue: 'https://api.uzbox.top',
        ),
      ),
    _client = client ?? HttpClient();

```

```
final Uri baseUri;
final HttpClient _client;
Future<Map<String, dynamic>> health() => _mapRequest('GET', '/health');
Future<Map<String, dynamic>> catalogReadiness() =>
    _mapRequest('GET', '/v1/catalog/readiness');
Future<List<Map<String, dynamic>>> catalogComponents({
    required String category,
    int limit = 100,
}) => _listRequest(
    'GET',
    '/v1/catalog/components',
    query: {'category': category, 'limit': '$limit'},
);
Future<List<Map<String, dynamic>>> catalogPrices({int limit = 500}) =>
    _listRequest('GET', '/v1/catalog/prices', query: {'limit': '$limit'});
Future<Map<String, dynamic>> buildOptions({
    required int budget,
    required String useCase,
    required List<String> gameCategories,
    required String direction,
    List<String> officeApps = const [],
    bool needsWirelessNetwork = false,
    String memorySize = '16GB',
    String storageSize = '512GB',
    bool noGpuBuild = false,
    String? ownedGpuModel,
}) => _mapRequest(
    'POST',
    '/v1/build/options',
    body: {
        'budget': budget,
        'use_case': useCase,
        'game_categories': gameCategories,
        'direction': direction,
        'office_apps': officeApps,
        'needs_wireless_network': needsWirelessNetwork,
        'memory_size': memorySize,
        'storage_size': storageSize,
        'no_gpu_build': noGpuBuild,
        'owned_gpu_model': ownedGpuModel,
    },
);
Future<Map<String, dynamic>> reviewText(String text) =>
    _mapRequest('POST', '/v1/review/analyze', body: {'text': text});
Future<Map<String, dynamic>> reviewImage(
    Uint8List bytes, {
    String contentType = 'image/jpeg',
}) async {
    final uri = baseUri.resolve('/v1/review/analyze/image');
    final boundary = 'UzBox-${DateTime.now().microsecondsSinceEpoch}';
```

```

try {
    final request = await _client
        .postUrl(uri)
        .timeout(const Duration(seconds: 15));
    request.headers.set(
        HttpHeaders.contentTypeHeader,
        'multipart/form-data; boundary=$boundary',
    );
    request.write('--$boundary\r\n');
    request.write(
        'Content-Disposition: form-data; name="image"; filename="config.jpg"\r\n',
    );
    request.write('Content-Type: $contentType\r\n\r\n');
    request.add(bytes);
    request.write('\r\n--$boundary--\r\n');
    final response = await request.close().timeout(
        const Duration(seconds: 30),
    );
    final text = await utf8.decoder.bind(response).join();
    final decoded = text.isEmpty ? null : jsonDecode(text);
    if (response.statusCode < 200 || response.statusCode >= 300) {
        final detail = decoded is Map<String, dynamic>
            ? decoded['detail']?.toString()
            : null;
        throw UzBoxApiException(
            detail ?? '图片分析失败（${response.statusCode}）',
            statusCode: response.statusCode,
        );
    }
    if (decoded is! Map<String, dynamic>) {
        throw const UzBoxApiException('服务器返回了无法识别的数据');
    }
    return decoded;
} on UzBoxApiException {
    rethrow;
} on TimeoutException {
    throw const UzBoxApiException('图片分析超时，请稍后重试');
} on SocketException {
    throw const UzBoxApiException('无法连接服务器，请检查网络');
}
}

Future<Map<String, dynamic>> estimatePerformance({
    required String cpuld,
    required String gpuld,
    required String resolution,
    required List<String> gamelds,
}) => _mapRequest(
    'POST',
    '/v1/perf/estimate',
    body: {

```

UzBox ai 装机助手 V1.0

```

    'hardware': {'cpu': cpuld, 'gpu': gpuld},
    'resolution': resolution,
    'games': gamelds,
  },
);
Future<Map<String, dynamic>> upgradePlan({
  required int budget,
  required Map<String, dynamic> current,
  required String need,
  required List<String> games,
  required String resolution,
  required int targetFps,
}) => _mapRequest(
  'POST',
  '/v1/upgrade/plan',
  body: {
    'budget': budget,
    'current': current,
    'need': need,
    'games': games,
    'resolution': resolution,
    'target_fps': targetFps,
  },
);
Future<Map<String, dynamic>> _mapRequest(
  String method,
  String path, {
    Map<String, String>? query,
    Object? body,
  }) async {
  final value = await _request(method, path, query: query, body: body);
  if (value is! Map<String, dynamic>) {
    throw const UzBoxApiException('服务器返回了无法识别的数据');
  }
  return value;
}
Future<List<Map<String, dynamic>>> _listRequest(
  String method,
  String path, {
    Map<String, String>? query,
    Object? body,
  }) async {
  final value = await _request(method, path, query: query, body: body);
  if (value is! List) {
    throw const UzBoxApiException('服务器返回了无法识别的数据');
  }
  return value.whereType<Map<String, dynamic>>().toList(growable: false);
}
Future<Object?> _request(
  String method,

```

```

UzBox ai 装机助手 V1.0

String path, {
  Map<String, String>? query,
  Object? body,
}) async {
  final uri = baseUrl.resolve(path).replace(queryParameters: query);
  try {
    final request = await _client
      .openUrl(method, uri)
      .timeout(const Duration(seconds: 15));
    request.headers.contentType = ContentType.json;
    request.headers.set(HttpHeaders.acceptHeader, ContentType.json.mimeType);
    if (body != null) request.write(jsonEncode(body));
    final response = await request.close().timeout(
      const Duration(seconds: 20),
    );
    final text = await utf8.decoder.bind(response).join();
    final decoded = text.isEmpty ? null : jsonDecode(text);
    if (response.statusCode < 200 || response.statusCode >= 300) {
      final detail = decoded is Map<String, dynamic>
        ? decoded['detail']?.toString()
        : null;
      throw UzBoxApiException(
        detail ?? '请求失败（${response.statusCode}）',
        statusCode: response.statusCode,
      );
    }
    return decoded;
  } on UzBoxApiException {
    rethrow;
  } on TimeoutException {
    throw const UzBoxApiException('连接服务器超时，请稍后重试');
  } on SocketException {
    throw const UzBoxApiException('无法连接服务器，请检查网络');
  } on FormatException {
    throw const UzBoxApiException('服务器返回了无法识别的数据');
  }
}

// File: flutter_app/lib/screens/primary_flows.dart
import 'dart:async';
import 'dart:math' as math;
import 'dart:typed_data';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import '../api/uzbox_api.dart';
import '../app_theme.dart';
import '../widgets/flow_components.dart';
class AiBuildScreen extends StatefulWidget {
  const AiBuildScreen({super.key});
  @override

```

```

State<AiBuildScreen> createState() => _AiBuildScreenState();
}

class _AiBuildScreenState extends State<AiBuildScreen> {
  int step = 0;
  double budget = 6850;
  int purpose = 0;
  bool ownedGpu = false;
  bool wireless = false;
  int buildPreference = 2;
  int colorPreference = 0;
  int upgradePreference = 0;
  int memoryPreference = 0;
  int storagePreference = 1;
  final selectedGames = <int>{};
  static const steps = ['预算和用途', '场景选择', '购买和外观', '补充偏好'];
  static const games = <({String title, String asset})>[
    (title: '什么都玩', asset: 'assets/images/game_pubg.png'),
    (title: '瓦罗兰特', asset: 'assets/images/game_valorant.png'),
    (title: 'CS2', asset: 'assets/images/game_cs2.png'),
    (title: 'PUBG', asset: 'assets/images/game_pubg.png'),
    (title: '三角洲行动', asset: 'assets/images/game_delta_force.png'),
    (title: '云顶之弈', asset: 'assets/images/game_tft.png'),
    (title: 'LOL', asset: 'assets/images/game_lol.png'),
    (title: 'COD', asset: 'assets/images/game_cod.png'),
    (title: '赛博朋克 2077', asset: 'assets/images/game_cyberpunk.png'),
    (title: '荒野大镖客 2', asset: 'assets/images/game_rdr2.png'),
    (title: 'GTA5', asset: 'assets/images/game_gta5.png'),
    (title: '黑神话悟空', asset: 'assets/images/game_wukong.png'),
    (title: '地平线 6', asset: 'assets/images/game_forza.png'),
    (title: '艾尔登法环', asset: 'assets/images/game_elden_ring.png'),
    (title: '城市天际线', asset: 'assets/images/game_cities.png'),
    (title: '我的世界', asset: 'assets/images/game_minecraft.png'),
  ];
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: AppTheme.background,
      body: SafeArea(
        bottom: false,
        child: Stack(
          children: [
            SingleChildScrollView(
              physics: const BouncingScrollPhysics(),
              padding: const EdgeInsets.fromLTRB(20, 8, 20, 104),
              child: Column(
                children: [
                  const _AiPageHeader(),
                  const SizedBox(height: 16),
                  _StepHeader(step: step, titles: steps),
                  const SizedBox(height: 14),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```



```

Align(
  alignment: Alignment.centerLeft,
  child: Container(
    width: 78,
    height: 26,
    padding: const EdgeInsets.symmetric(horizontal: 10),
    alignment: Alignment.center,
    decoration: BoxDecoration(
      color: Colors.white.withValues(alpha: 0.72),
      borderRadius: BorderRadius.circular(13),
    ),
    child: Text(
      '第 ${step + 1}/4 步',
      style: const TextStyle(
        fontSize: 11,
        fontWeight: FontWeight.w700,
      ),
    ),
  ),
  child: const SizedBox(height: 11),
  AnimatedSwitcher(
    duration: const Duration(milliseconds: 220),
    child: _stepContent(),
  ),
],
),
),
Align(
  alignment: Alignment.bottomCenter,
  child: UzBottomAction(
    title: step == 3 ? '生成配置方案' : '下一步',
    showBack: step > 0,
    onBack: () => setState(() => step -= 1),
    onPressed: () {
      if (step < 3) {
        setState(() => step += 1);
      } else {
        Navigator.of(context).push(
          uzRoute(
            AiGeneratingScreen(
              budget: budget.round(),
              useCase: const ['游戏', '办公', '游戏兼办公'][purpose],
              gameCategories: selectedGames
                .map((index) => games[index].title)
                .toList(),
              direction: const [
                'fps',
                'aaa',
                'balanced',

```

```
      ][buildPreference],
      needsWirelessNetwork: wireless,
      memorySize: memoryPreference == 0 ? '16GB' : '32GB',
      storageSize: const [
        '512GB',
        '1TB',
        '2TB',
      ][storagePreference],
    ),
  ),
);
}
},
),
),
],
),
),
);
}
Widget _stepContent() {
  if (step == 0) {
    return UzSoftCard(
      key: const ValueKey(0),
      padding: const EdgeInsets.fromLTRB(22, 22, 22, 12),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text(
            '预算和用途',
            style: TextStyle(fontSize: 22, fontWeight: FontWeight.w700),
          ),
          const SizedBox(height: 7),
          const Text(
            '先确定大方向，AI 会按预算控制配置。',
            style: TextStyle(color: AppTheme.secondary, fontSize: 14),
          ),
          const SizedBox(height: 14),
          Row(
            children: [
              const Text(
                '预算范围',
                style: TextStyle(fontSize: 14, fontWeight: FontWeight.w800),
              ),
              const Spacer(),
              Text(
                '¥ ${budget.round()}',
                style: const TextStyle(
                  fontSize: 22,
                  fontWeight: FontWeight.w900,
```

```
    ),  
    ),  
  ],  
),  
const SizedBox(height: 8),  
const Row(  
  children: [  
    Text(  
      '¥ 3000',  
      style: TextStyle(color: AppTheme.secondary, fontSize: 11),  
    ),  
    Spacer(),  
    Text(  
      '¥ 30000',  
      style: TextStyle(color: AppTheme.secondary, fontSize: 11),  
    ),  
  ],  
),  
const SizedBox(height: 8),  
Row(  
  children: [  
    _RoundStepButton(  
      icon: Icons.remove_rounded,  
      onTap: () => setState(  
        () => budget = (budget - 500).clamp(3000, 30000),  
      ),  
    ),  
    Expanded(  
      child: SizedBox(  
        height: 34,  
        child: SliderTheme(  
          data: SliderTheme.of(context).copyWith(  
            activeTrackColor: AppTheme.primary,  
            inactiveTrackColor: const Color(0xFFE1E4E7),  
            thumbColor: Colors.white,  
            trackHeight: 5,  
            thumbShape: const RoundSliderThumbShape(  
              enabledThumbRadius: 16,  
            ),  
            overlayShape: SliderComponentShape.noOverlay,  
          ),  
          child: Slider(  
            min: 3000,  
            max: 30000,  
            value: budget,  
            onChanged: (value) => setState(() => budget = value),  
          ),  
        ),  
      ),  
    ),  
  ],  
),
```

```
_RoundStepButton(  
  icon: Icons.add_rounded,  
  onTap: () => setState(  
    () => budget = (budget + 500).clamp(3000, 30000),  
  ),  
),  
],  
),  
const SizedBox(height: 14),  
const Text(  
  '主要用途',  
  style: TextStyle(fontSize: 14, fontWeight: FontWeight.w800),  
)  
,  
const SizedBox(height: 8),  
UzSegmentedControl(  
  options: const ['游戏', '办公', '游戏兼办公'],  
  selected: purpose,  
  onSelected: (value) => setState(() => purpose = value),  
)  
,  
const SizedBox(height: 14),  
Row(  
  children: [  
    const Expanded(  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          Text(  
            '无显卡方案',  
            style: TextStyle(  
              fontSize: 14,  
              fontWeight: FontWeight.w700,  
            ),  
          ),  
          SizedBox(height: 4),  
          Text(  
            '是否自备显卡',  
            style: TextStyle(  
              color: AppTheme.secondary,  
              fontSize: 11,  
            ),  
          ),  
        ],  
      ),  
    ),  
    SizedBox(  
      height: 44,  
      child: Switch.adaptive(  
        value: ownedGpu,  
        activeTrackColor: AppTheme.primary,  
        onChanged: (value) => setState(() => ownedGpu = value),  
      ),  
    ),  
  ],  
)
```

```

    ),
    ),
  ],
),
],
),
);
}
if (step == 1) {
  return Column(
    key: const ValueKey(1),
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        '选择你常玩的游戏',
        style: TextStyle(fontSize: 22, fontWeight: FontWeight.w700),
      ),
      const SizedBox(height: 5),
      const Text(
        '可多选，AI 会按游戏需求调整配置',
        style: TextStyle(color: AppTheme.secondary, fontSize: 14),
      ),
      const SizedBox(height: 16),
      const Text(
        '全部游戏',
        style: TextStyle(fontSize: 14, fontWeight: FontWeight.w700),
      ),
      const SizedBox(height: 10),
      GridView.builder(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        itemCount: games.length,
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
          crossAxisCount: 4,
          crossAxisSpacing: 8,
          mainAxisSpacing: 14,
          childAspectRatio: 0.82,
        ),
        itemBuilder: (context, index) {
          final selected = selectedGames.contains(index);
          final game = games[index];
          return InkWell(
            onTap: () => setState(() {
              selected
                ? selectedGames.remove(index)
                : selectedGames.add(index);
            }),
            borderRadius: BorderRadius.circular(13),
            child: Column(
              children: [

```

```
Expanded(
  child: Stack(
    clipBehavior: Clip.none,
    children: [
      Positioned.fill(
        child: ClipRRect(
          borderRadius: BorderRadius.circular(13),
          child: index == 0
            ? const _AllGamesArtwork()
            : Image.asset(game.asset, fit: BoxFit.cover),
        ),
      ),
    ],
    if (selected)
      Positioned(
        right: -4,
        top: -4,
        child: Container(
          width: 22,
          height: 22,
          decoration: const BoxDecoration(
            color: Colors.black,
            shape: BoxShape.circle,
          ),
          child: const Icon(
            Icons.check_rounded,
            color: Colors.white,
            size: 14,
          ),
        ),
      ),
    ],
  ),
  const SizedBox(height: 6),
  Text(
    game.title,
    maxLines: 1,
    overflow: TextOverflow.fade,
    style: const TextStyle(
      fontSize: 10.5,
      fontWeight: FontWeight.w600,
    ),
  ),
],
),
);
},
),
];
);
```